```
# Create a virtual environment
python -m venv .venv
 # or py -m venv .venv
# Activate it
.venv\Scripts\Activate
# Install OpenCV
 python -m pip install opencv-python
# Install NumPy
python -m pip install numpy
 # or install both together
python -m pip install numpy opencv-python
```

P1 Load image, display, convert to gray, resize and save output image

```
import cv2
img = cv2.imread('input.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
resized = cv2.resize(gray, (500, 400))
cv2.imshow('Result', resized)
cv2.imwrite('output.jpg', resized)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

P2 Colour space conversion between RGB, Grayscale, and HSV

```
import cv2
img = cv2.imread("nature.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
cv2.imshow("Original Image", img)
cv2.imshow("Grayscale Image", gray)
cv2.imshow("HSV Image", hsv)
print("RGB Pixel Value:", img[100,100])
print("Gray Pixel Value:", gray[100,100])
print("HSV Pixel Value:", hsv[100,100])
cv2.waitKey(0)
cv2.destroyAllWindows()
```

P3 Adjust Brightness and Contrast Smoothing Filters (Gaussian and Median)

```
import cv2
img = cv2.imread("nature.jpg")
alpha = 1.5   # Contrast factor (greater than 1 increases contrast)
beta = 50     # Brightness value (positive values increase brightness)
result = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)
cv2.imshow("Original Image", img)
cv2.imshow("Brightness and Contrast Adjusted", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

P4 Apply Smoothing Filters (Gaussian and Median)

```
import cv2
img = cv2.imread("nature.jpg")
gaussian = cv2.GaussianBlur(img, (5, 5), 0)
median = cv2.medianBlur(img, 5)
cv2.imshow("Original Image", img)
cv2.imshow("Gaussian Filter", gaussian)
cv2.imshow("Median Filter", median)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

P5 Sobel and Canny Edge Detection Techniques

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("building.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)
sobel = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)
plt.figure(figsize=(15, 5))
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")
plt.subplot(1, 4, 2)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")
plt.subplot(1, 4, 3)
```

```
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")
plt.subplot(1, 4, 4)
plt.imshow(sobel, cmap="gray")
plt.title("Combined Sobel")
plt.axis("off")
plt.show()
```

P6 Detect Corners using Harris Corner Detector

```
import cv2
import numpy as np

image = cv2.imread("building.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)
corners = cv2.cornerHarris(gray, 2, 3, 0.04)
corners = cv2.dilate(corners, None)
image[corners > 0.01 * corners.max()] = [0, 0, 255]
cv2.imshow("Harris Corner Detection", image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

P7 Feature Detection & Matching Using ORB (Oriented FAST and Rotated BRIEF)

```
import cv2
import matplotlib.pyplot as plt
img1 = cv2.imread("image1.jpg")
img2 = cv2.imread("image2.jpg")
if img1 is None or img2 is None:
    print("Error: Could not load one or both images.")
    exit()
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
orb = cv2.ORB_create(nfeatures=1000)
keypoints1, descriptors1 = orb.detectAndCompute(gray1, None)
keypoints2, descriptors2 = orb.detectAndCompute(gray2, None)
print("Keypoints in Image 1:", len(keypoints1))
print("Keypoints in Image 2:", len(keypoints2))
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(descriptors1, descriptors2)
matches = sorted(matches, key=lambda x: x.distance)
best_matches = matches[:50]
result = cv2.drawMatches(
    img1, keypoints1,img2,keypoints2,best_matches,None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS,
)
plt.figure(figsize=(16, 8))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("ORB Feature Detection and Matching")
plt.axis("off")
plt.show()

print("Total matches:", len(matches))
print("Best matches displayed:", len(best_matches))
```

P8 Image Segmentation Using Thresholding and KMeans Clustering

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
image = cv2.imread("image.jpg")
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
pixels = image_rgb.reshape((-1, 3))
pixels = np.float32(pixels)
K = 3

criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
_, labels, centers = cv2.kmeans(
    pixels, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS
)

centers = np.uint8(centers)
segmented_data = centers[labels.flatten()]
segmented_image = segmented_data.reshape(image_rgb.shape)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(segmented_image)
plt.title("K-Means Segmentation")
plt.axis("off")
plt.show()
```

P9 Implement face detection using Haar Cascade Classifier

```
import cv2

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

    for x, y, w, h in faces:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    cv2.imshow("Face Detection", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

P10 Motion Detection / Object Tracking

Perform background subtraction or simple object tracking

```
import cv2
cap = cv2.VideoCapture(0)
background_subtractor = cv2.createBackgroundSubtractorMOG2()
while True:
    ret, frame = cap.read()
    if not ret:
        break
    foreground_mask = background_subtractor.apply(frame)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    foreground_mask = cv2.morphologyEx(foreground_mask, cv2.MORPH_OPEN, kernel)
    contours, _ = cv2.findContours(
        foreground_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )
    for contour in contours:
        if cv2.contourArea(contour) < 500:
            continue
        x, y, w, h = cv2.boundingRect(contour)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(
            frame,"Motion Detected",(x, y - 10),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0, 255, 0),2,
        )
    cv2.imshow("Motion Detection", frame)
    cv2.imshow("Foreground Mask", foreground_mask)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```